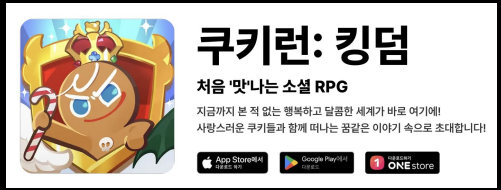


Argo Workflows를 활용해 사내 배포도구를 개발한 경험

박진수 (LinkedIn)

발표자 소개

- 쿠키런 IP 기반 게임들로 유명한 데브시스터즈 인프라팀 DevOps Engineer 만 4년
- 입사 초기
  - Kubernetes, AWS, Terraform을 활용한 인프라 운영
  - 각종 오픈소스를 조합한 자동화, 시스템 구축
  - 신규 게임 런칭
- 최근 2년 반
  - 간간히 Flan이라는 사내 배포 플랫폼 개발해왔음
  - 2026년에 들어서야 업무 시간의 50%정도를 투자 중



오늘 하고싶은 얘기

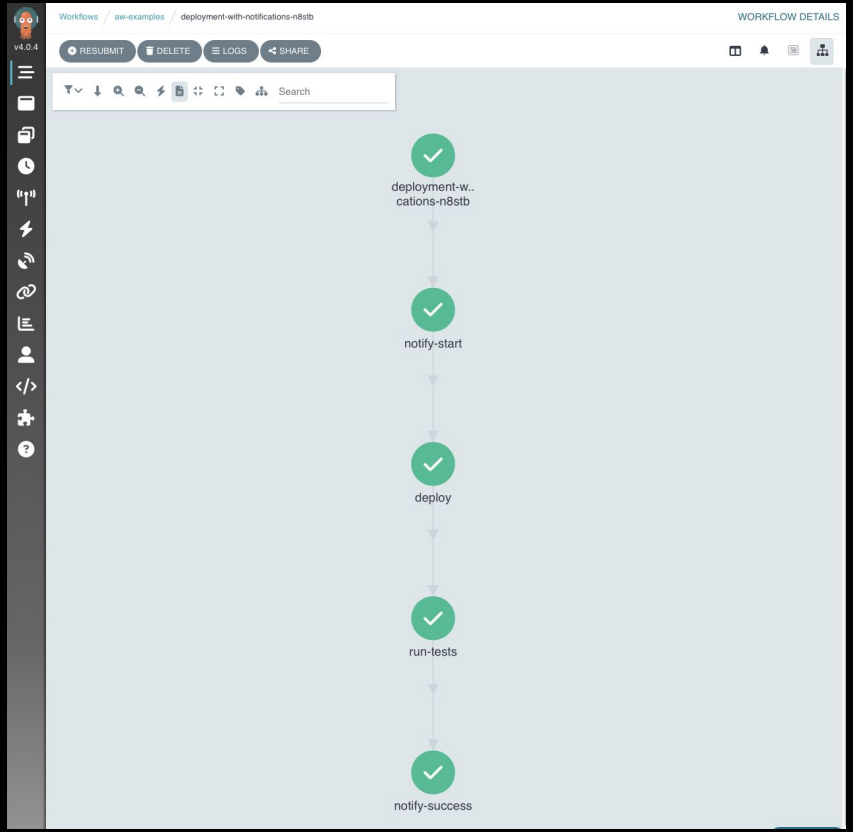
- Argo workflows에 대한 소개
- 저희 제품은 왜 argo workflows를 사용해왔는지
- 컨테이너 기반 워크플로우의 장점
- 실제로 사용해보니 어땠는지
- 컨테이너 기반이라 오히려 존재하는 단점과 그에 대한 대응 방안
- Q&A

Argo Workflows에 대한 소개

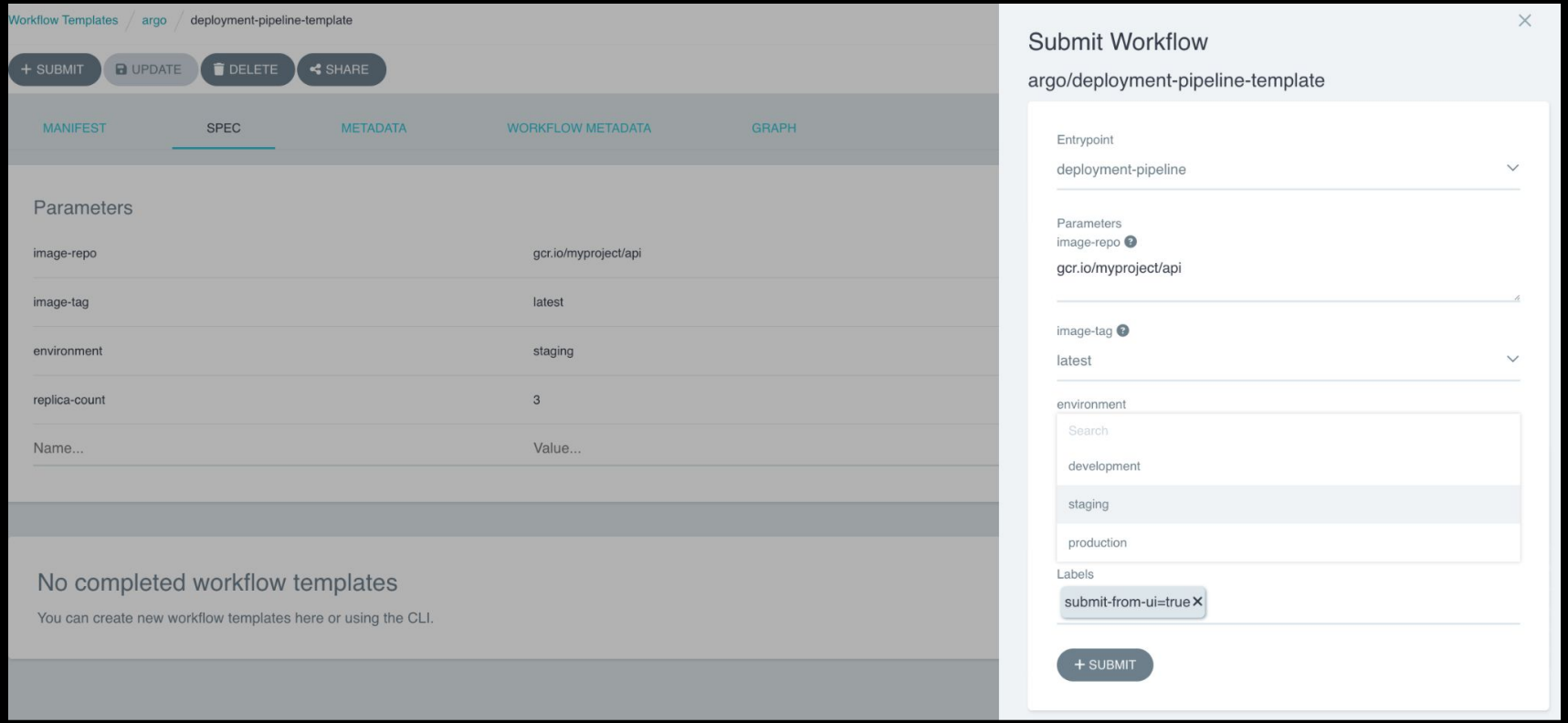
Argo Workflows

- Argoproj의 시초. 자세한 역사는 다큐멘터리 영상 추천
- Container가 workflow의 실행 단위
- K8s Native - 대부분의 설정, 동작이 k8s resource와 k8s api 활용
- Components
  - argo-server
  - workflow-controller
- Step - init, main, wait 컨테이너로 구성.

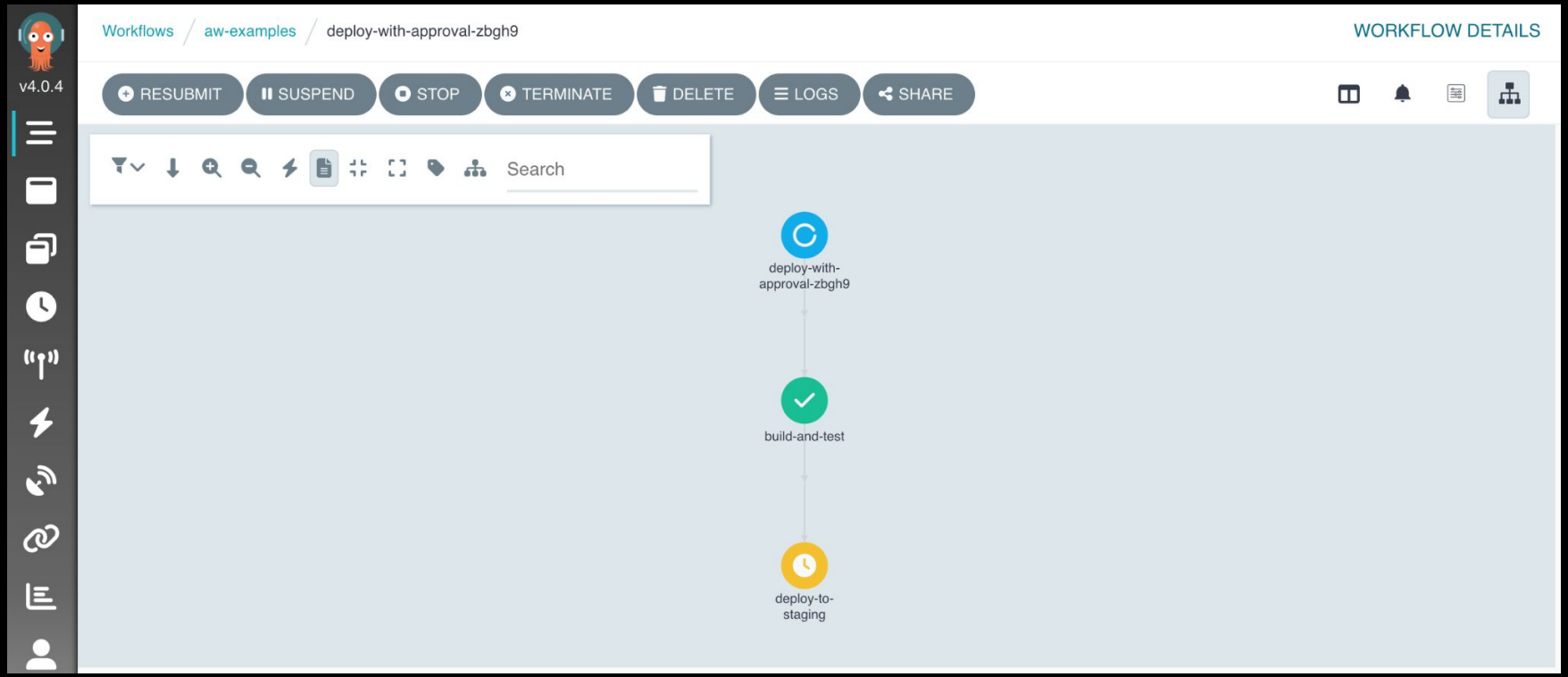
기능 살펴보기 - 각 스텝이 하나의 Pod



기능 살펴보기 - 재사용 가능한 Workflow Template

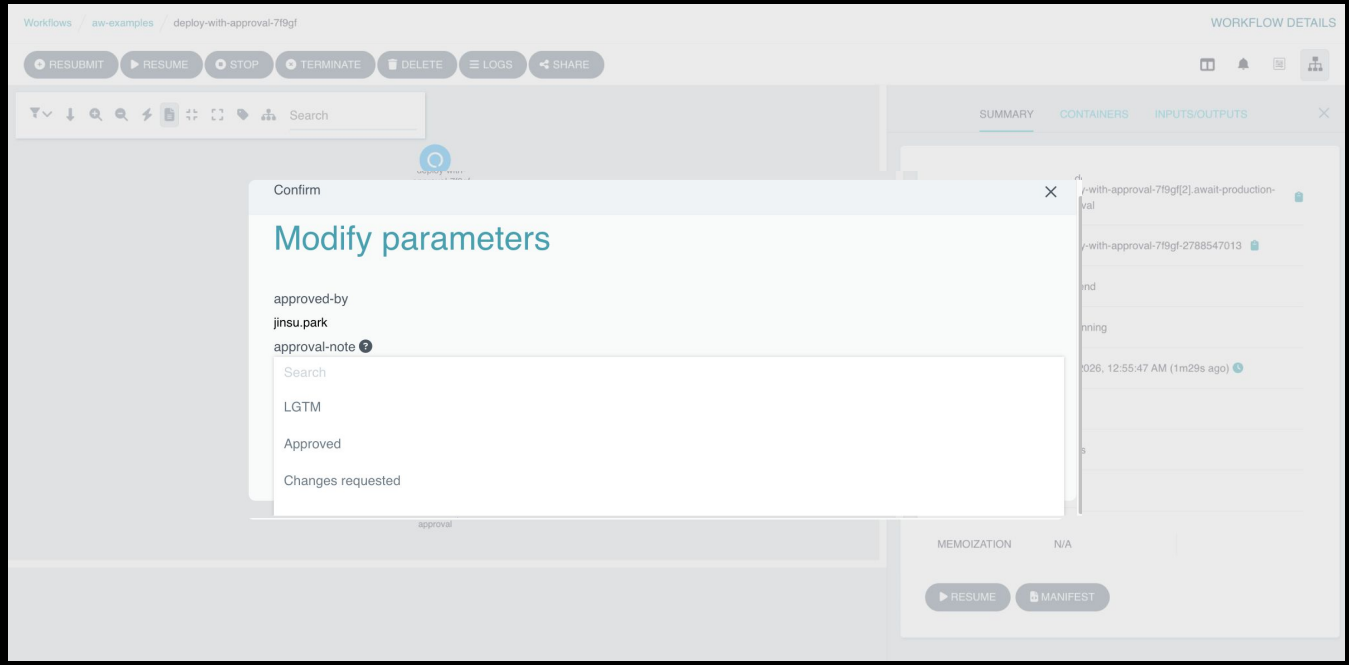


기능 살펴보기 - 일시정지, 승인



기능 살펴보기 - 일시정지, 승인

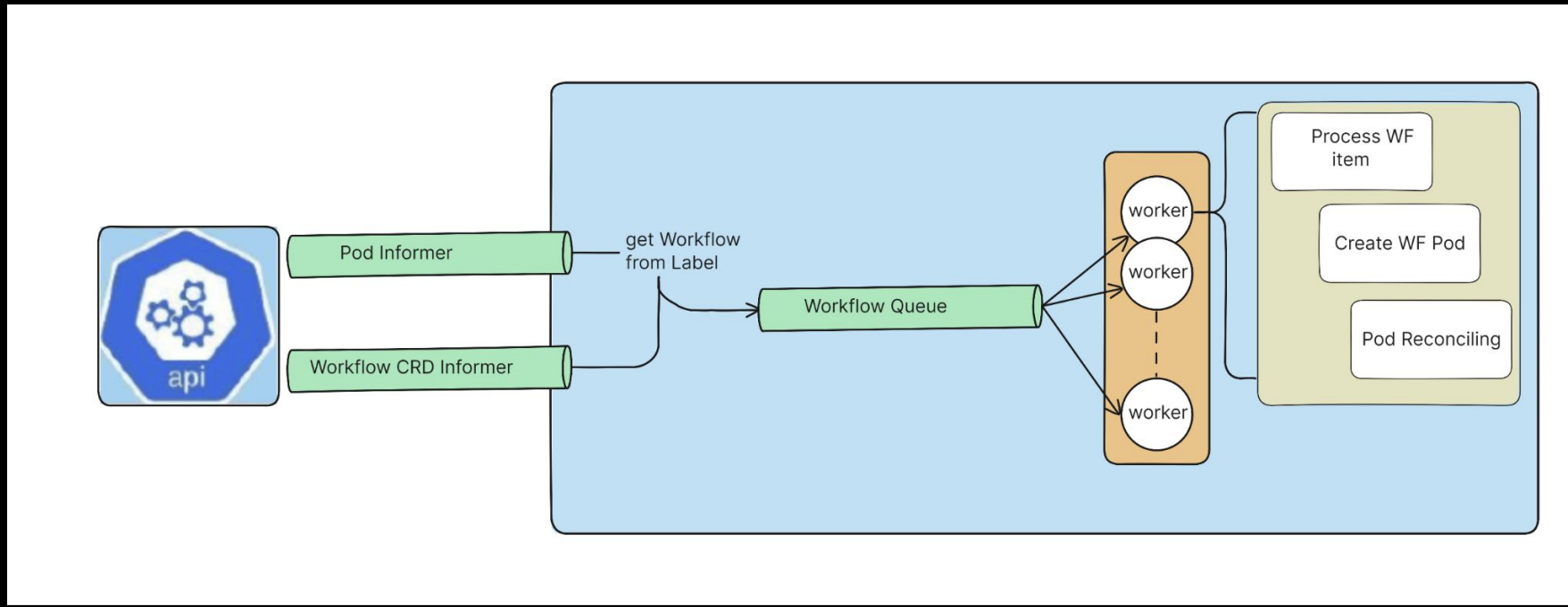




Argo Workflows Manifest 예시

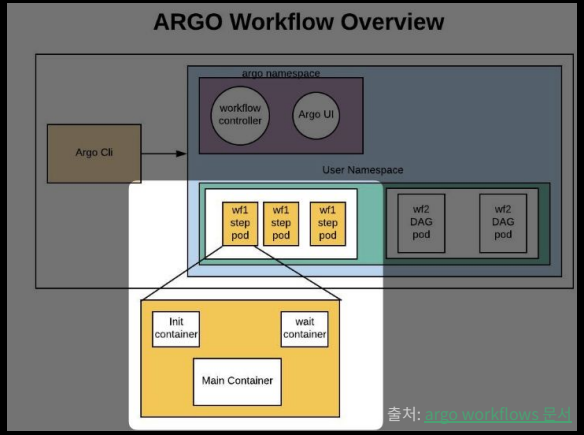
```yaml
apiVersion: argoproj.io/v1alpha1
kind: Workflow
metadata:
  generateName: hello-world-
spec:
  entrypoint: hello-world
  templates:
    - name: hello-world
      container:
        image: busybox
        command: [ echo ]
        args: [ "hello world" ]
```

K8s Reconciliation



출처: https://argo-workflows.readthedocs.io/en/release-4.0/architecture/

Executor가 구현하는 Step의 구성



Init

- initContainer
- Artifact 다운로드
- Parameter interpolation
- Container image: argoexec image

Main

- **실제 컨테이너 수행**

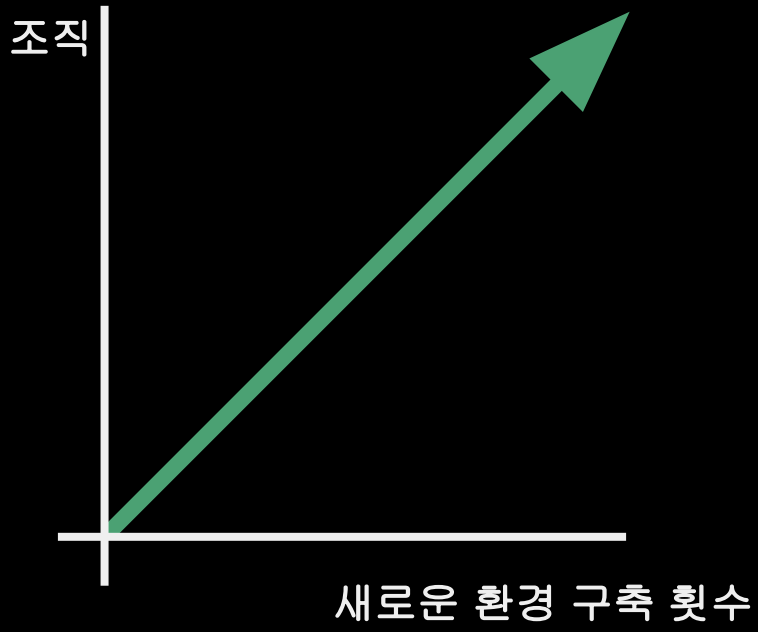
Wait

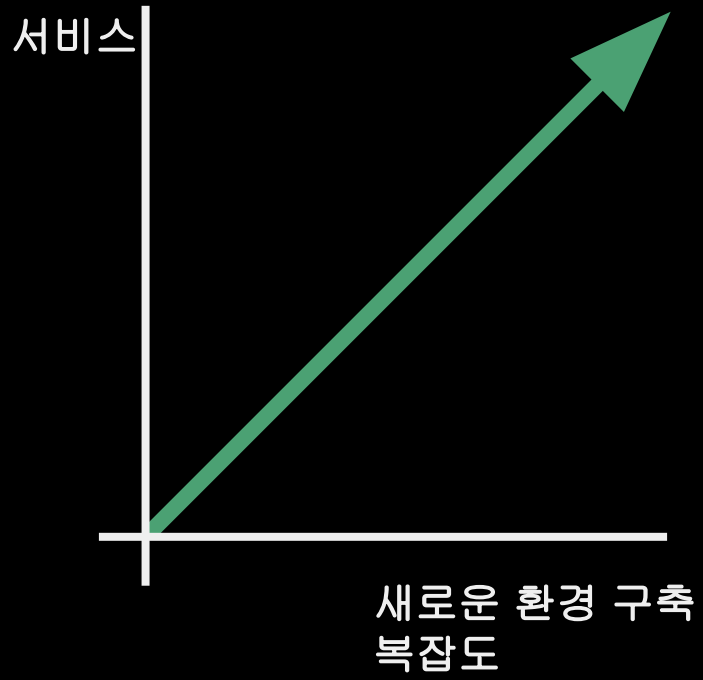
- 사이드카. main 컨테이너와 시작은 동시
- main 컨테이너의 종료 감시
- Artifact 업로드
- Parameter 수집 및 보고

Argo Workflows를 감싼 내부 배포 플랫폼, Flan

- 목표
  - DEV) 개발자 및 QA 엔지니어, 기획자도 k8s 환경에 손쉽게 배포하는 플랫폼
  - PROD) 프로덕션 배포에 필요한 작업을 쉽고 안전하게 지원하는 플랫폼
  - 개인적 상상 - 현재의 목표는 환경 생성, 서비스 배포에 초점이 잡혀있지만 추후에는 좀 더 다양한 기능을 지원하는 워크플로우 제품이 될 수 있으려나

그냥 오픈소스 배포 도구를 사용할 수는 없었는가?





다음과 같은 작업들이 자동화될 수 있어야함

- 애플리케이션 배포 전 DB 마이그레이션 적용
- 손쉬운 카나리 배포 및 롤백
- 배포 파이프라인의 각 단계를 수행하기 전 예상되는 동작을 확인 후 승인
- K8s resource 배포가 아닌 동작 실행(e.g., 내부 API 호출)
- 원하는 커스텀 로직 실행 - 임의의 script

-> ***단순 배포 도구가 아닌 워크플로우 도구가 필요하겠다.***

그냥 오픈소스 워크플로우 도구를 사용할 수는 없었는가?

- 전사에서 자주 사용되는 작업들을 추상화하여 스텝으로 제공하고 싶었음.
- 각 제품들은 스텝을 필요에 따라 자유롭게 조립하여 사용할 수 있도록 하고 싶었음.
- **다만 워크플로우 도구들은 너무 general하다.**
  우리 조직을 위해 opinionated 되어있지 않아 결국 사용하기 불편하고 human error에 취약함.

우리 조직에서 필요로 했던 추가적인 기능들

- 배포 시점에 선택 가능한 옵션을 실시간으로 조회하여 드롭다운 선택지로 제공 (e.g., ECR 이미지 목록, git branch 목록)
- Prod 환경에 대한 파괴적 액션 금지 및 Prod 환경에 대한 높은 식별성
- 새로운 배포에 대한 예상 Diff 제공
- 배포된 서비스에 대한 카탈로그 혹은 디스커버리
- 일시정지 및 승인
- Multi-cluster
- …

우리 조직에서 필요로 했던 추가적인 기능들

드롭다운 선택지
배포 시점에 선택 가능한 옵션을 조회하여 드롭다운 선택지로 제공

Diff 제공
현재 상태를 기준으로 새로운 배포 시 적용될 예상 Diff 제공

Prod 환경 구분
파괴적 액션 금지 및 Prod 환경에 대한 높은 식별성

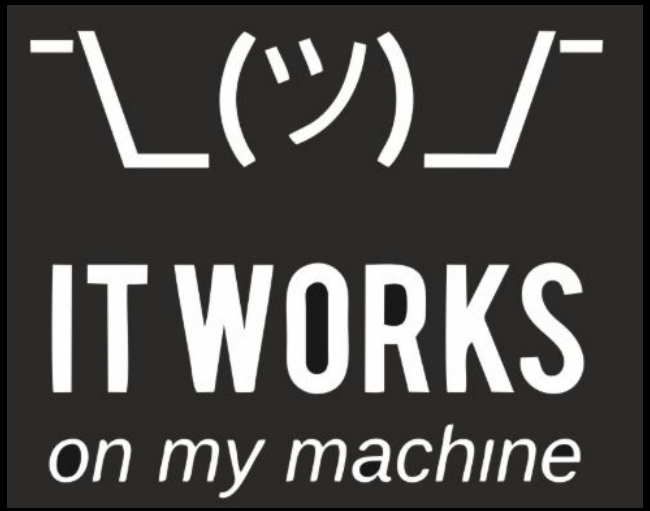
카탈로그 / 디스커버리
배포된 서비스에 대한 정보, 메타데이터 제공

일시정지 / 승인 / 재개
워크플로우 실행 도중 일시 정지하여 승인이 완료될 때까지 대기

Multi-Cluster
여러 클러스터에 대한 배포를 한 곳에서 관리

컨테이너 기반 워크플로우의 장점

재현 가능성



컨테이너 기반 workflow의 대조군:

worker 서버가 요청을 받아 일련의 작업을 수행하는 방식

재현 가능성



컨테이너 기반 workflow의 대조군:

worker 서버가 설정을 받아 일련의 작업을 수행하는 방식

재현 가능성

- 의존성 고정
  - 의존하는 패키지들이 이미 이미지 빌드 시점에 컨테이너에 고정됨.
  - 특히 terraform, helm, gRPC, kubectl, db client 등의 인프라 도구들은 버전 변경 시의 사이드 이펙트가 클 수 있음.
- 개발자의 로컬이든, CI 머신이든, 클라우드의 서버이든 같은 이미지라면 동일한 실행 환경이 재현된다.

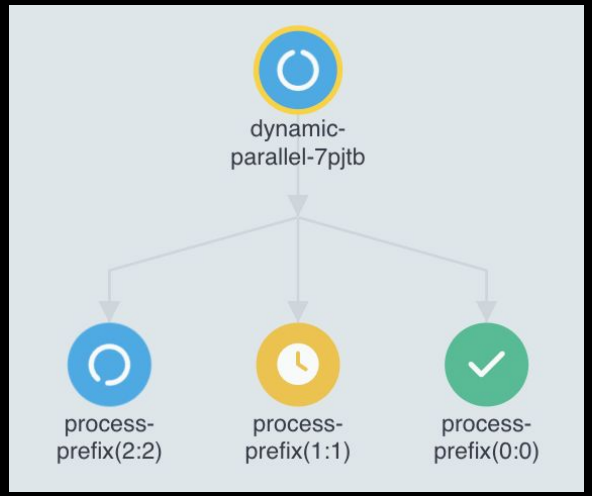
-> **불변성(immutability)으로부터 오는 재현 가능성(reproducibility)!**

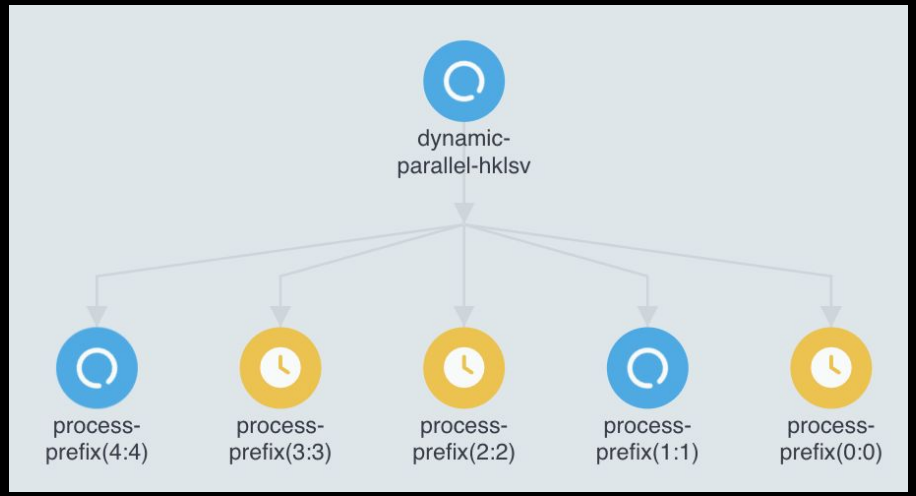
격리

- 독립된 실행 환경
  - 특정 작업을 위한 컨테이너의 패키지 업그레이드, 설정 변경이 다른 작업을 위한 컨테이너에 영향을 끼치지 않는다.
  - step 별로 다른 의존성을 갖도록 할 수 있다.
  - 동일 step이라도 이미지 버전을 나누어 서로 버전의 의존성을 지원할 수도 있다.
    - db-migration:1.0.2-mysql-5.7
    - db-migration:1.0.2-mysql-8.0
- CPU, 메모리 등의 리소스 격리
  - 무거운 작업이 있더라도 그것은 그 컨테이너 내부의 일일 뿐.

리소스를 탄력적으로 사용할 수 있다.

- 워크플로우 내의 각 스텝들의 cpu, memory 등의 자원이 상이한 경우.
- 워크플로우의 진행 과정에서 필요로하는 자원의 양이 달라지는 경우.
- 조회한 데이터에 따라 처리에 필요한 자원이 상이한 경우.





Non-컨테이너 기반 레거시 vs 컨테이너 기반

[Before] Non-컨테이너 기반 레거시

1. 데이터 조회
2. 처리
3. 집계

- 전체가 한 덩어리
  -> 완료된 worker 리소스 반환 불가
- 데이터 크기와 무관하게 서버는 항상 떠있다.

[After] 컨테이너 기반 워크플로우

1. 데이터 조회
2. 동적으로 Pod 개수 결정
3. 처리
   Pod 1
   Pod 2
   Pod N
4. 집계

- 종료 즉시 Pod 삭제 및 자원 반환
- 데이터 크기에 따라 동적으로 container N개 할당

분산 처리가 쉽다.

- 단일 노드로 해결하기 힘든 규모의 작업
- 인프라적 문제 해결을 위해 여러 노드가 필요한 작업

-> K8s의 생태계를 이용해 손쉽게 오토스케일하여 수행 가능.

배포 플랫폼에서는 활용하지 못한 장점들

- 탄력적 리소스 사용, 쉬운 분산 처리는 “컨테이너” 기반의 큰 장점.
- 하지만, 내부 배포 플랫폼은 데이터 처리, 머신러닝 등과 달리
  이런 부분에서의 니즈는 크지 않았다.
- 각 스텝이 오히려 되게 가벼운 편.
- 그치만 Argo Workflows의 기능이 풍부하고, kubernetes native한 면이 잘 맞는다.
- -> Why not 느낌.

컨테이너 기반 워크플로우를 사용하며 체감한 단점

단점 1. 컨테이너 실행으로 인한 오버헤드가 크다. 느리다.

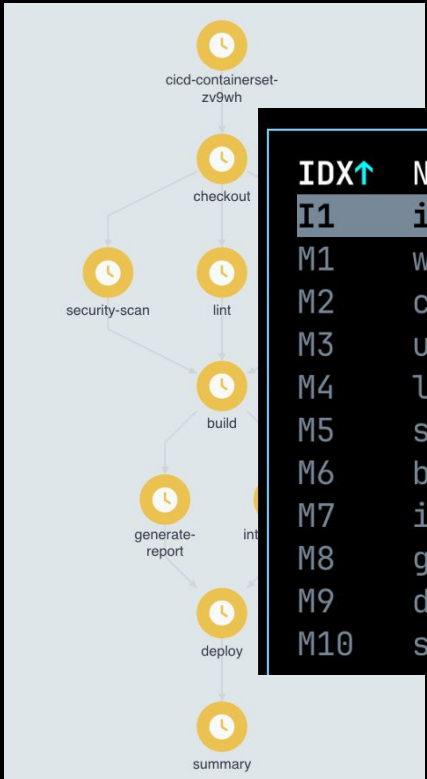
왜 느린가? (컨테이너 소모임이니까 언급은 해봄. 자세한 내용은 생략..)

- Pod 생성 시 k8s control plane 측 처리
- Karpenter/cluster-autoscaler의 node scale out, node join
- Pod 시작 - CNI의 처리, image pull
- 항상 깨끗한 상태의 container. 캐시도 없다.

해결책 - Container Set 사용하기

- 여러 스텝을 하나의 Pod 안에 몰아넣기.
- 여러 컨테이너가 동일한 이미지를 사용하는 경우 - 이미지 pull은 한 번만.
- 병렬, 순차적 실행 모두 가능

해결책 - Container Set 사용하기

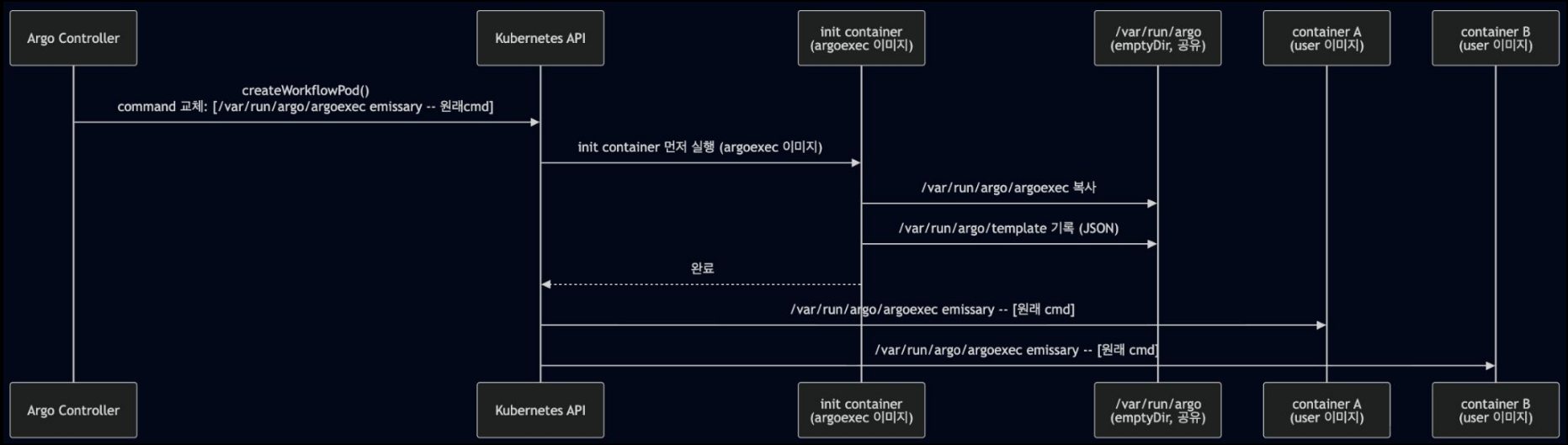


containers(aw-examples/cicd-containerset-dd4dq)[11]

IDX↑	NAME	PF	IMAGE	READY	STATE	REST
I1	init	●	quay.io/argoproj/argoexec:v4.0.4	true	Completed	
M1	wait	●	quay.io/argoproj/argoexec:v4.0.4	false	Completed	
M2	checkout	●	busybox	false	Completed	
M3	unit-test	●	busybox	false	Completed	
M4	lint	●	busybox	false	Completed	
M5	security-scan	●	busybox	false	Completed	
M6	build	●	busybox	false	Completed	
M7	integration-test	●	busybox	false	Completed	
M8	generate-report	●	busybox	false	Completed	
M9	deploy	●	busybox	false	Completed	
M10	summary	●	busybox	false	Completed	

해결책 - Container Set 사용하기

대조군: 일반적인 step이나 DAG는 왜 느린가?



해결책 - Container Set 사용하기

실험군: Container Set은 왜 빠른가?

+ 🧐 어떻게 한 Pod 내에서 컨테이너를 순차적으로 실행하는가?

사실 Executor는 main 컨테이너의 command도 바꿔치기함

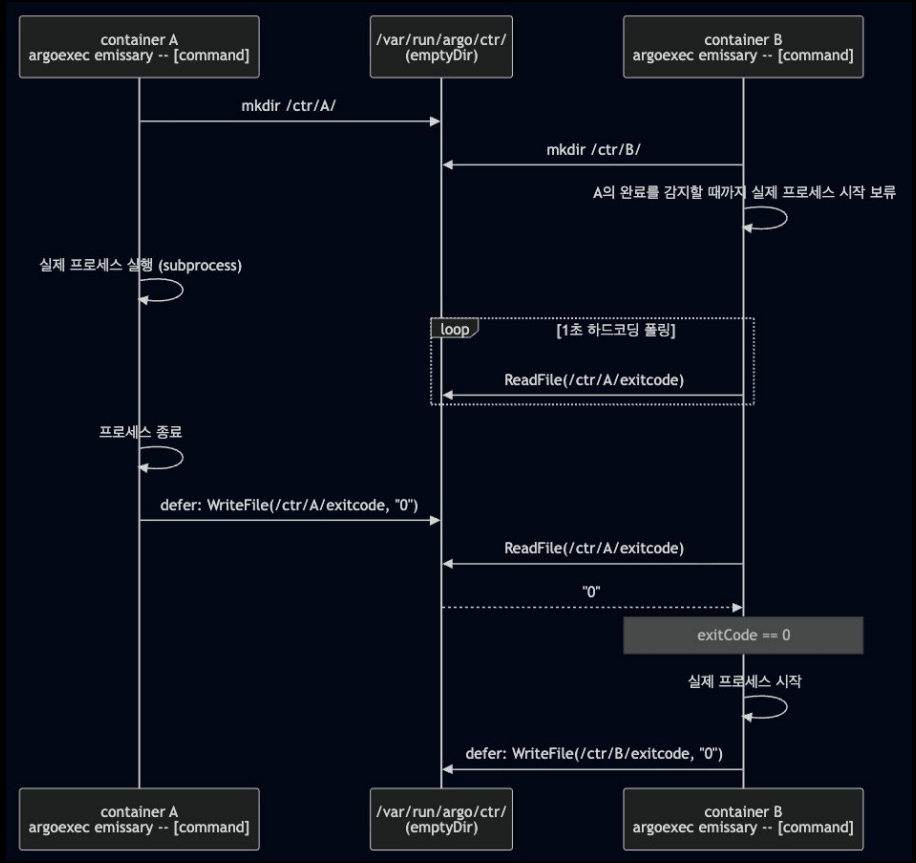
```yaml
apiVersion: argoproj.io/v1alpha1
kind: Workflow
metadata:
  generateName: hello-world-
spec:
  entrypoint: hello-world
  templates:
    - name: hello-world
      container:
        image: busybox
        command: [ echo ]
        args: [ "hello world" ]
```

```yaml
- args:
  - hello world
  command:
  - /var/run/argo/argoexec
  - emissary
  - --loglevel
  - info
  - --log-format
  - text
  - --gloglevel
  - "0"
  - --
  - echo
```

NAME	PF	IMAGE	READY	STATE
init	●	quay.io/argoproj/argoexec:v4.0.4	true	Completed
wait	●	quay.io/argoproj/argoexec:v4.0.4	false	Completed
main	●	busybox	false	Completed

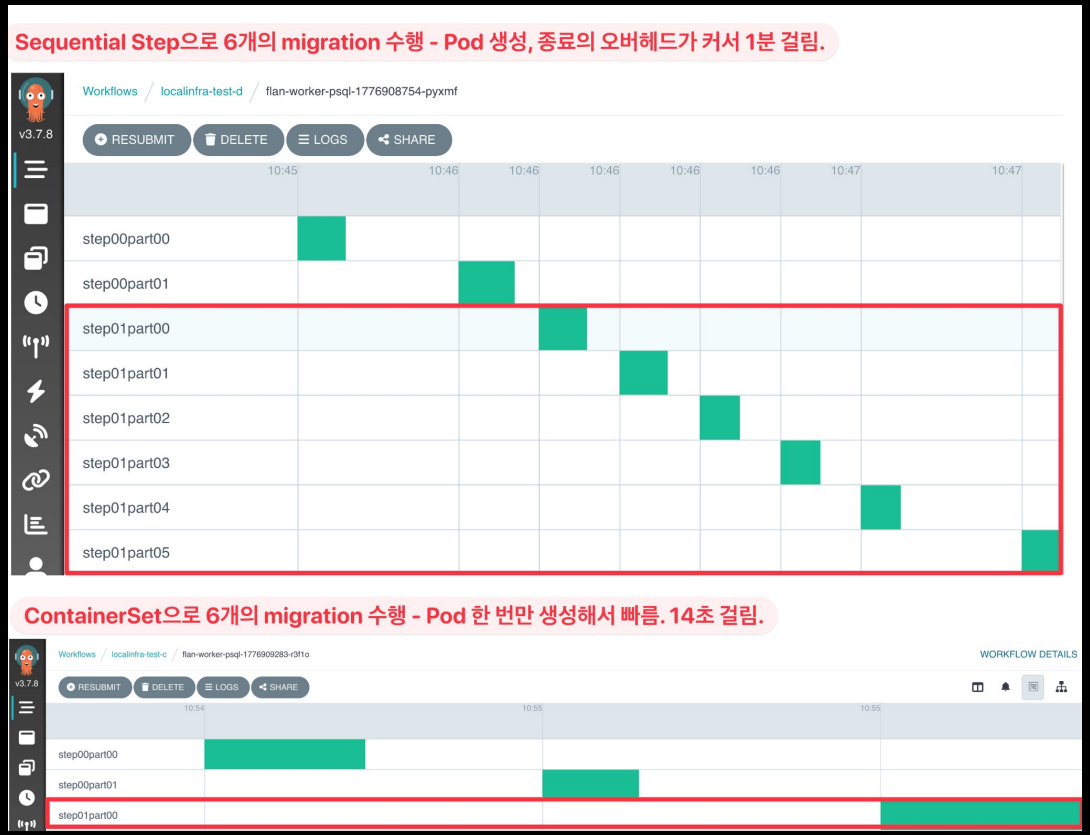
- Pod 내의 컨테이너들은 동시에 시작
- 컨테이너의 command는 왜 바꿔치기하는 걸까?
- argoexec binary는 어디서 오는가? 뭘하는 녀석인가?

해결책 - Container Set 사용하기




```
/var/run/argo # tree
.
├── argoexec
├── ctr
│   ├── build
│   │   └── exitcode
│   ├── checkout
│   │   └── exitcode
│   └── deploy
│       └── exitcode
```

해결책 - Container Set 사용하기



6개의 가벼운 작업을 수행하는데

1분 -> 15초로 **75% 감소**

해결책 - Executor Plugin 기능 활용하기

- Executor plugin이란 하나의 workflow 실행 동안
  임의의 http 서버에게 step의 실행, 결과를 요청할 수 있도록 하는 plugin
- 작업이 매우 간단하면서도 잦은 경우에 사용하면 좋다.
  - 어디에 알림 보내기
  - 내부 API 호출하기
- 컨테이너 기반 step과 plugin 기반 step을 혼합할 수도 있음.

느리다에 대한 기타 해결책

- 컨테이너 이미지 미리 Pull 받아두기
- 작은 Node를 over provisioning 해두기
- ...

단점 2. Step간 데이터 전달이 느리고 번거롭다.

- GitHub Actions에서 step간 데이터 전달과 유사한 불편.
- 프로그래밍 언어에서는 한 함수의 return 값을 다른 함수의 인자로 사용하는 게 참 쉬움.
- 반면 컨테이너 간의 데이터 공유는 복잡하고 번거롭다.

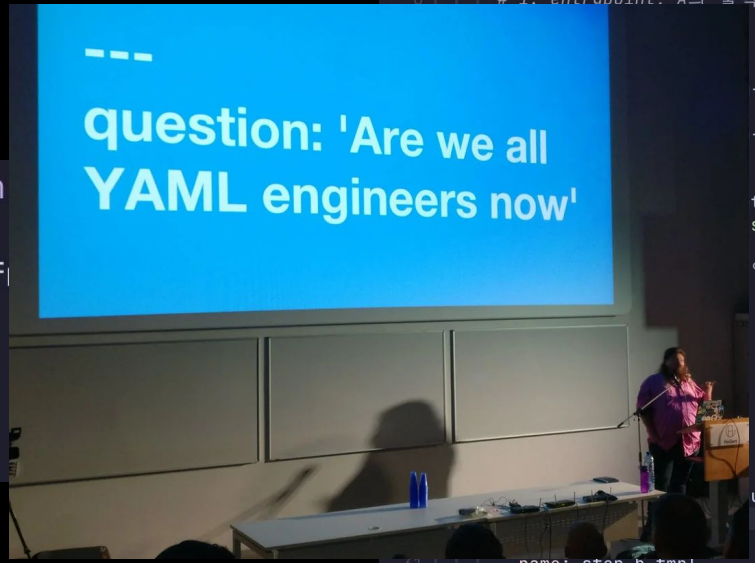
단점 2. Step간 데이터 전달이 느리고 번거롭다.



```
func A() string {return

func B(greet string) {f

func main() {
    B(A())
}
```




```
apiVersion: argoproj.io/v1alpha1
kind: Workflow
spec:
  entrypoint: main
  templates:
    # 1. entrypoint: A의 출력을 B의 입력으로 직접 연결해줘야 함


-tmpl

-tmpl


t
steps.step-a.outputs.parameters.result}}"

일로 써야 함 + outputs 선언


> /tmp/result.txt"]  # 파일에 써야 전달 가능


ult.txt  # 어느 경로인지 선언

 + 표현식으로 참조
    - name: step-b-tmpl
      inputs:
        parameters:
          - name: greet             # 받을 파라미터 선언
      container:
        image: alpine
        command: [sh, -c]
        args: ["echo {{inputs.parameters.greet}}"]  # 표현식으로 주입
```

단점 2. Step간 데이터 전달이 느리고 번거롭다.

- 느리다 - Pod 간의 데이터 공유가 어렵다보니 보통 S3에 artifact 업로드/다운로드 함으로써 데이터를 공유하기 마련.
- 번거롭다 - Yaml로 manifest를 작성하는 작업의 난이도가 높음.
  - DRY, 추상화, 테스트 코드 활용 등등이 어렵다.

‘느리다’의 해결책

업로드, 다운로드를 하지 않은 채 공유할 수 있도록 해보자.

- ContainerSet을 통해 같은 Pod 내의 컨테이너끼리는 emptyDir 활용
- PVC

‘번거롭다’의 해결책 - 프로그래밍 언어 활용

- 사실 workflow 라는 custom resource를 제출하는 것은 workflow execute api를 호출하는 것과 별반 다를 것 없다.
  - workflow manifest = execute api의 http request body
- Yaml이 아닌 프로그래밍 언어나 DSL을 통해 manifest를 만들자.
  - 기대효과: 풍부한 기능을 바탕으로 더 안전하고 쉽게 manifest를 만들 수 있다.
  - 이런 시도가 발전하다보면 내부 CLI 도구, 내부 플랫폼 제품이 될 수도.
- 궁극적으로는 **argo workflows는 엔진일 뿐!**
  겉제품으로 많은 추상화 제공하면 됨.

번외) Hera - Workflow를 Python으로 작성하기

Hera at a glance

```python
from hera.workflows import DAG, Workflow, script


# Turn a function into a reusable "Script template"
# using the script decorator
@script()
def echo(message: str):
    print(message)


# Orchestration logic lives *outside* of business logic
with Workflow(
    generate_name="dag-diamond-",
    entrypoint="diamond",
) as w:
    with DAG(name="diamond"):
        A = echo(name="A", arguments={"message": "A"})
        B = echo(name="B", arguments={"message": "B"})
        C = echo(name="C", arguments={"message": "C"})
        D = echo(name="D", arguments={"message": "D"})
        A >> [B, C] >> D  # Define execution order

# Create the workflow directly on your Argo Workflows cluster!
w.create()
```

https://github.com/argoproj-labs/hera

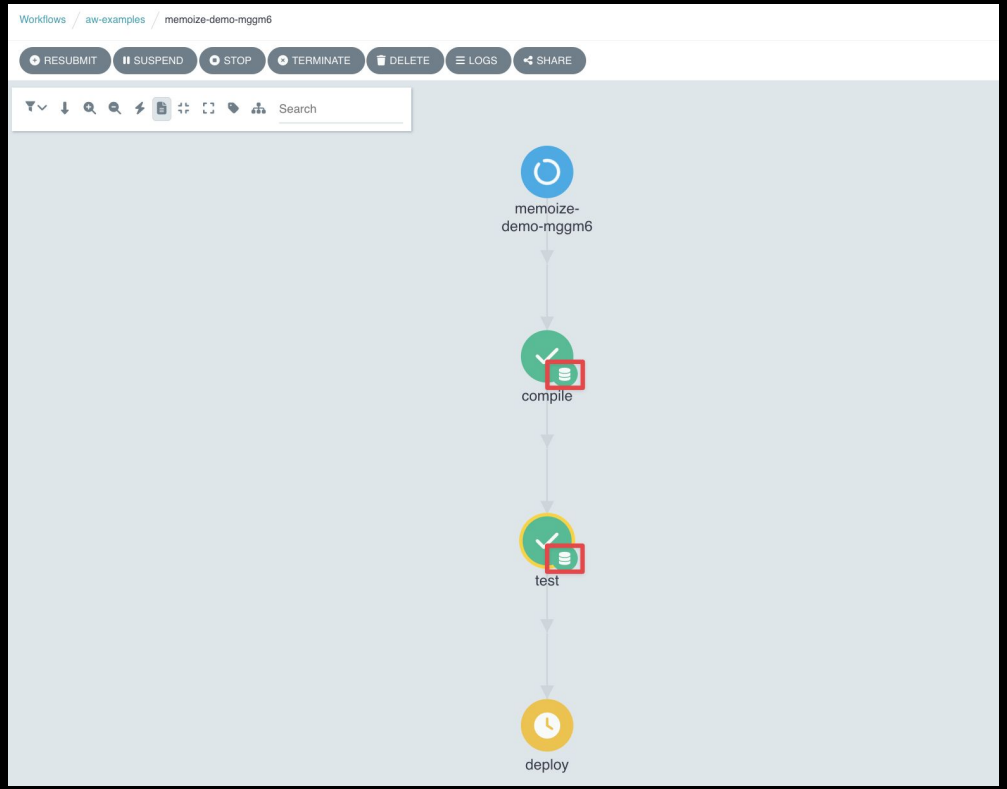
단점 3. 재실행, 재현이 힘들고 불편하다.

- 재현 가능성이 컨테이너의 장점이었으나 현실은..
  몇가지 전제가 만족되어야한다.
  - 작업 자체의 멱등성
  - 원자성
- 컨테이너만 재현된다고 작업 자체가 재현되는 게 아니다.
- 멱등성이나 원자성이 보장되지 않으면? -> 실패한 스텝만 재실행하기
  - 문제) 컨테이너간의 의존성. 컨테이너 A를 실행해야 그 output을 B가 사용.

해결책

- 멱등성, 원자성 보장하거나
- 컨테이너간의 의존성을 최대한 없애고 선택적 실행 가능하게 하기.
  - ‘진짜 이 컨테이너들간에 의존성이 필요한가?’ 고민해보기.
  - 결정론적으로 미리 값을 알 수 있도록 하자.
- 이전에 성공한 스텝은 캐시된 결과 활용하기
  - Argo Workflows의 memoization 유용함.
    스텝 간 의존성을 없애고 결과값을 workflow manifest 바깥에서 조회하는 것

해결책 예시 - Memoization



이전에 성공했던 스텝은 캐시된 결과를 이용하도록 하고 스킵할 수 있다.

단점 4. K8s에 사용자는 관심 없는 K8s Resource가 배포됨.

pods(bookcellar)[18]

NAME↑	PF	READY	STATUS	RESTARTS
bookcellar-65d7f48d47-rqhst	●	1/1	Running	0
bookcellar-sync-29614450-jjc5b	●	0/1	Completed	0
bookcellar-sync-29614460-r6xmw	●	0/1	Completed	0
bookcellar-sync-29614470-hfwbv	●	0/1	Completed	0
cicd-containerset-4zg49	●	0/10	Pending	0
cicd-containerset-6f5x6	●	0/10	PodInitializing∆	0
cicd-containerset-ffvfg	●	0/10	Init:0/1	0
cicd-containerset-hgxth	●	0/10	Init:0/1	0
cicd-containerset-r4zbf	●	0/10	Pending	0
deploy-with-approval-qcpzw-run-command-1241259498	●	0/2	Completed	0
deployment-with-notifications-5ssgn-1340600742-agent	●	0/1	Error	2↑
deployment-with-notifications-8hcgk-1340600742-agent	●	0/1	Error∆	2↑
deployment-with-notifications-8qjsr-1340600742-agent	●	1/1	Running	1
deployment-with-notifications-79k99-1340600742-agent	●	0/1	Error∆	2↑
deployment-with-notifications-hj57g-1340600742-agent	●	0/1	Error	1
deployment-with-notifications-jbbcp-1340600742-agent	●	0/1	Error	2↑
deployment-with-notifications-pbcph-1340600742-agent	●	1/1	Running	1
deployment-with-notifications-whjqq-1340600742-agent	●	0/1	Error	2↑

- 워크플로우로 수행하는 작업이 주요 기능이면 괜찮음.
- 하지만 배포 자동화 같은 부가적인, 추상화된 작업이라면 좀 곤란함.

정리

- Argo Workflows는 컨테이너를 실행 단위로 하는 워크플로우 엔진
- 막상 써보면 단점들도 느껴지긴한다.
- 단점을 완화하는 방법들:
  - Container set, executor plugin, memoization, artifact 등 argo workflows가 제공하는 기능들을 활용해 완화
  - 컨테이너 기반이고 k8s에서 실행되는 것을 전제로 하기에 일반적인 k8s에서의 최적화 기법들 적용 가능